

Junio 5, 2026

Documento de Especificación de Requerimientos

CASO: Voz del
Ciudadano-VozCi

Integrantes:

- Andrew Owim
- Inga Rojas

Tabla de Contenidos

1. Introducción.....	2
1.1 Propósito del sistema.....	2
1.2 Alcance del sistema.....	2
1.3 Descripción General del sistema.....	2
1.3.1 Contexto del sistema.....	2
1.3.2 Funciones del sistema.....	3
1.3.3 Características del usuario.....	3
1.3.4 Limitaciones.....	4
1.4 Definiciones.....	4
2. Referencias.....	4
3. Requerimientos del sistema.....	4
3.1 Interfaces externas.....	5
3.2 Requerimientos Funcionales.....	8
3.3 Requerimientos de usabilidad.....	8
3.4 Requerimientos de rendimiento.....	9
3.5 Requerimientos de bases de datos.....	9
3.6 Restricciones de diseño.....	9
3.7 Atributos del sistema de software.....	10
4. Verificación.....	10
4.1 Interfaces externas.....	10
4.2 Requerimientos Funcionales.....	11
4.3 Requerimientos de usabilidad.....	11
4.4 Requerimientos de rendimiento.....	12
4.5 Requerimientos de bases de datos.....	12
4.6 Restricciones de diseño.....	12
4.7 Atributos del sistema de software.....	12
5. Apéndices.....	13

1. Introducción

1.1 Propósito del sistema

El Legislativo de la República requiere el desarrollo de la plataforma digital "Voz del Ciudadano" para automatizar el procesamiento de Iniciativas Legislativas de los Ciudadanos , hasta alcanzar el limite de 25,000 firmas digitales.

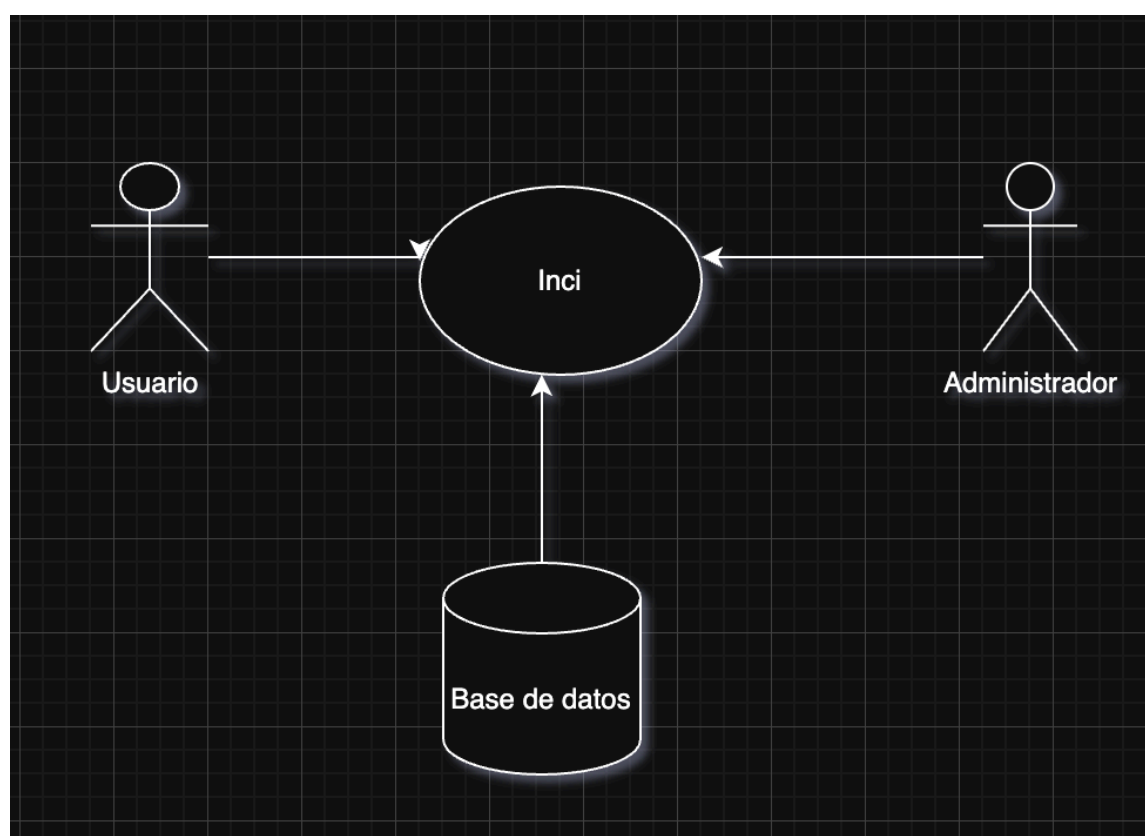
1.2 Alcance del sistema

VozCi es un sistema basado en tecnología web orientada a digitalizar a recolectar propuestas normativas y firmas

El sistema se limitará a la implementación de una plataforma web funcional accesible desde navegadores web que implemente lo anteriormente mencionado.

1.3 Descripción General del producto

1.3.1 Perspectiva del producto



a) Interfaces del sistema

El sistema VozCi interactúa con una base de datos relacional, utilizado para almacenar información estructurada relacionada a entidades de dominio y opcionalmente un servicio de almacenamiento de objetos dedicado a guardar datos no estructurados, como imágenes y archivos multimedia.

b) Interfaces de usuario

VozCi ofrece una interfaz web con la que los usuarios podrán interactuar para registrar iniciativas ciudadanas . Asimismo, los administradores podrán acceder usando una interfaz web a ver lo que

registraron los ciudadanos

c) Interfaces de software

VozCi es un sistema stand-alone. No requiere de integraciones con otros servicios, a excepción de una base de datos relacional, la cual es contemplada como parte de la implementación nativa del sistema.

d) Comunicaciones de interfaces

El sistema núcleo de VozCi expone una interfaz RESTful a las distintas interfaces de usuario, utilizando comunicación compatible con los protocolos HTTP/HTTPS, con datos siendo intercambiados en formato JSON. Además, el sistema maneja la autenticación sin guardar estado alguno, por lo que cada petición al sistema debe estar acompañada por un token de sesión válido suministrado por el sistema.

e) Memoria

Las restricciones de memoria de almacenamiento de VozCi son sumamente dependientes del tamaño o la cantidad de usuarios que vayan a reportar sus incidencias. Para un correcto desempeño del sistema se recomienda 4GB de memoria RAM en el lado del servidor.

f) Operaciones

El sistema opera como una aplicación web, la cual es accesible a través de navegadores web comunes. Cualquier ciudadano podrá dejar su incidencia.

g) Requisitos de adaptación del sitio.

El sistema debe ser posible de desplegar en entornos estándares de servidores que soportan aplicaciones web, con configuraciones mínimas, como configuraciones sobre la conexión a la base de datos, en caso éstas cambien. No se requiere ningún tipo de hardware adicional aparte de un servidor estándar en el lado del servidor y dispositivos con un navegador web en el lado del cliente.

1.3.2 Funciones del producto

VozCi provee funcionalidades que soportan el registro de iniciativas . entre sus principales funciones se encuentran :

- Poder ver las anteriores reportes de los usuarios y ver si algún ciudadano ya a registrado lo que iba a reportar uno.
- Ver por parte del administrador todos las iniciativas que se enviaron.

1.3.3 Características del usuario

El sistema está orientado para los siguientes tipos de usuarios:

- Ciudadanos : personas que desean notificar alguna iniciativa
- Administradores del sistema: Personas con habilidades técnicas avanzadas, responsables de la configuración y mantenimiento del sistema, así como la resolución de problemas.
- Autoridades: miembros del legislativo de la república.

1.3.4 Limitaciones

VozCi está restringido a las siguientes limitaciones:

- El sistema debe ser desarrollado como una aplicación web compatible con navegadores actuales.
- El sistema debe usar una base de datos relacional para entidades de dominio, y almacenamiento de objetos para guardar información adicional.
- El sistema debe implementarse siguiendo una arquitectura RESTful.
- El sistema debe operar en ambientes de servidores estándares sin hardware especializado.
- El sistema no incluye integración de forma nativa con servicios de terceros

1.4 Definiciones

- **VozCi:** Voz del ciudadano.
- **API RESTful (VozCi API):** Interfaz expuesta por el sistema VozCi que permite la comunicación entre las interfaces de usuario y el backend mediante el uso de protocolos HTTP/HTTPS y formato JSON.

- **Token de sesión (JWT):** Credencial generada por el sistema VozCi para autenticar usuarios de forma sin estado (stateless) en cada solicitud al sistema.

2. Referencias

- IEEE
IEEE 29148-2011 - Systems and Software Engineering – Life Cycle Processes – Requirements Engineering.
2011.
https://nirmt.com/storage/uploads/E-BOOK_BE-INDUSTRIAL-AND-SAFETY/29148-2011%20-%20ISOIEC%20International%20Standard%20-%20Systems%20and%20software%20engineering%20--%20Life%20cycle%20processes%20--Requirements.pdf
- Ian Sommerville
Software Engineering (10th GLOBAL Edition).
Pearson Education, 2016
<https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf>

3. Requerimientos del sistema

3.1 Interfaces externas

Definir todas las entradas y salidas del sistema VozCi. Cada interfaz complementa lo descrito en la sección 1.3.1.

3.1.1 Interfaces del sistema

- **INT-01: API RESTful del sistema Inci**
 - a) **Nombre:** VozCi API
 - b) **Descripción:** Servicios del sistema a interfaces web. Gestiona autenticación sin estado (stateless) mediante tokens de sesión.
 - c) **Fuente/Destino:** Entrada: Peticiones HTTP desde el frontend. Salida: Respuestas JSON con datos, códigos de error y registros en el historial de operaciones.
 - d) **Rango/Exactitud:** Token de sesión: JWT válido con expiración configurable. Respuestas: Códigos HTTP estándar (200, 201, 400, 401, 403, 404, 500). Número de llamadas limitadas por cliente, en caso de realizar varias en corto periodo de tiempo.
 - e) **Unidades:** JSON (UTF-8) sobre HTTP/HTTPS.
 - f) **Timing:** Asíncrono, orientado a eventos HTTP. Contar con un tiempo de respuesta menor a 1 segundo para búsquedas, sin congestión frente a operaciones generales bajo gran carga de usuarios.
 - g) **Relaciones:** Es consumida por las interfaces INT-02 e INT-03. Lee y escribe datos en la interfaz INT-04.
 - h) **Formato de pantalla:** No comprende UI.
 - i) **Formato de ventana:** No comprende UI.
 - j) **Formato de datos:** Esquemas de datos para llamadas request/reponse en JSON.
 - k) **Formato de comandos:** Verbos RESTful, para la creación de endpoints que devuelvan registros de libros, usuarios, permiten modificaciones de catálogo y de registros de préstamos.
 - l) **Mensajes de fin:** Códigos de estado.

3.1.2 Interfaces de usuario

- **INT-02: Portal web del cliente**
 - a) **Nombre:** Portal del usuario Inci.
 - b) **Descripción:** Página principal que permite a los usuarios registrar sus incidencias.
 - c) **Fuente/Destino:** Entrada: Usuario desde navegador web. Salida: Lista de incidencias, estado de su incidencias , etc.
 - d) **Rango/Exactitud:** Búsqueda: Admisible entre 1 a 255 caracteres. Comentarios: Máximo de 1000 caracteres. Descripciones: Máximo de 2000 caracteres.
 - e) **Unidades:** Texto (UTF-8), documentos de comprobantes de alquiler de libros.
 - f) **Timing:** Respuesta de búsqueda y carga inicial menor a 2 segundos.
 - g) **Relaciones:** Depende de las interfaces INT-01 (RESTful) para obtener y enviar datos de la biblioteca y de INT-04 para generar solicitudes que modifican la base de datos.
 - h) **Formato de pantalla:** Diseño responsive con barra de búsqueda superior, listado de resultados paginados en tarjetas, ficha de detalles de libros con botón de reserva.
 - i) **Formato de ventana:** Ventanas de página principal, catálogo, detalle de libro, mi perfil, historial de préstamos, registro e inicio de sesión.
 - j) **Formato de datos:** Datos enviados al backend en JSON via HTTP/HTTPS junto a comprobantes de alquiler PDF.
 - k) **Formato de comandos:** Acciones precisas “Buscar”, “Ver detalle” e “Iniciar sesión”.

- l) **Mensajes de fin:** “Incidencia registrada con éxito”, “No tiene ningún registro aun”.
- **INT-03: Panel web del administrador.**
 - a) **Nombre:** Portal de administración Inci.
 - b) **Descripción:** Página de gestión que permite a encargados administradores de añadir, editar y eliminar las incidencias registras por los usuarios.
 - c) **Fuente/Destino:** Entrada: Usuario administrador o bibliotecario autenticado. Salida: Confirmaciones de operaciones, reportes estadísticos, listado de usuarios, etc.
 - d) **Rango/Exactitud:** ISBN: Formato estándar de 10 a 13 dígitos. Campos de texto: Máximo de 2000 caracteres.
 - e) **Unidades:** Texto (UTF-8), imágenes (JPEG/PNG)
 - f) **Timing:** Respuestas del servidor a menos de 3 segundos bajo carga de grandes registros tanto de usuarios, libros y registros junto a sus distintas operaciones pertinentes.
 - g) **Relaciones:** Depende de INT-01 (API RESTful). Las operaciones CRUD consultan a la interfaz INT-04 para verificar estados de usuarios o libros.

3.1.3 Interfaces de hardware

No aplica, ya que VozCi opera sobre hardware de servidor estándar sin requerir widgets u objetos adicionales.

3.1.4 Interfaces de software

- **INT-04: Base de datos relacional**
 - a) **Nombre:** Base de Datos Relacional de VozCi (INT-04)
 - b) **Descripción:** Componente de software encargado de almacenar y recuperar de forma persistente todas las entidades de dominio del sistema: iniciativas , imágenes si se adjunta . Es el único repositorio de datos estructurado del sistema.
 - c) **Fuente/Destino:** Fuente: capa de lógica de negocio del backend (API RESTful, INT-03). Destino: misma capa, como respuesta a consultas y operaciones de escritura. No es accedida directamente por ninguna interfaz de usuario.
 - d) **Rango/Exactitud:** ISBN: exactamente 10 o 13 dígitos numéricos. Historial de transacciones: retenido por un mínimo de 5 años. Integridad referencial: un registro de libro no puede eliminarse si existen préstamos activos asociados. Relación entre Recurso Bibliográfico y Ejemplares: 1:N obligatoria.
 - e) **Timing:** Sincrónico respecto a las peticiones de la API. El 95% de las consultas de búsqueda deben resolverse en menos de 1.5 segundos. Las operaciones de escritura (préstamos, devoluciones) deben completarse de forma atómica para evitar estados inconsistentes.
 - f) **Relaciones:** Es accedida exclusivamente por INT-01 (API RESTful). Las operaciones de eliminación están condicionadas por las restricciones de integridad que protegen a INT-02 e INT-03 de recibir datos huérfanos. El estado de disponibilidad de un libro es actualizado automáticamente al confirmarse un préstamo desde INT-02.
 - g) **Formato de pantalla:** No comprende UI.
 - h) **Formato de ventana:** No comprende UI.
 - i) **Formato de datos:** Tablas relacionales SQL que permitan el almacenamiento de entidades , incidencias , imagenes etc.

- j) **Formato de comandos:** Contar con operaciones SQL tradicionales para la correcta funcionalidad de respuestas a las llamadas de la API RESTful.
- k) **Mensajes de fin:** No comprende mensajes que no sean manejados por la API RESTful.

- **INT-05: Servicio de almacenamiento de objetos**

- a) **Nombre:** Servicio de almacenamiento de objetos para Inci
- b) **Descripción:** Componente encargado de almacenar datos no estructurados del sistema, principalmente recursos multimedia como imágenes de portadas u otros relacionados con recursos.
- c) **Fuente/Destino:** Fuente: administrador que sube imágenes desde INT-02 (panel de administración), enrutado a través de INT-01 (API). Destino: URL pública o firmada devuelta a INT-02 e INT-03 para renderizar las portadas en la interfaz web.
- d) **Rango/Exactitud:** Formatos aceptados: JPEG y PNG únicamente. Archivo de tamaño máximo como 5MB. Nombres de archivo: únicos, generados por el sistema para evitar colisiones (ej. UUID + extensión).
- e) **Unidades:** Archivos binarios (JPEG/PNG). URLs de acceso en texto plano (UTF-8). Tamaño medido en kilobytes/megabytes.
- f) **Timing:** Asíncrono respecto a la operación del catálogo: la carga de una imagen no bloquea el registro de incidencia en la base de datos. La URL del objeto queda disponible una vez completada la carga.
- g) **Formato de pantalla:** No comprende UI.
- h) **Formato de ventana:** No comprende UI.
- i) **Formato de datos:** Objetos binarios por llave única, con encabezados específicos para definir el tipo del contenido a dar (image/jpeg, image/png), size, upload_date. Su URL es usada como referencia dentro de la base de datos como fuente de acceso.
- j) **Formato de comandos:** Operaciones estándar de uso NoSQL para la gestión y recuperación de archivos multimedia.
- k) **Mensajes de fin:** No comprende mensajes, solo códigos de estado.

3.1.5 Interfaces de comunicación

- **INT-06: Protocolo HTTP/HTTPS con intercambio JSON**

- a) **Nombre:** Interfaz de comunicaciones HTTP/HTTPS para Inci.
- b) **Descripción:** Define el canal de comunicación y el protocolo de transferencia de datos entre las interfaces de usuario y la API RESTful . Establece reglas de transporte de datos, seguridad de la conexión y formato de intercambio de mensajes.
- c) **Fuente/Destino:** Fuente: Cualquier cliente web (navegador de usuario o administrador) que origine una petición. Destino: Servidor que aloja la API RESTful de Inci. El canal es bidireccional, las respuestas siguen el camino inverso.
- d) **Rango/Exactitud:** Solo se aceptan únicamente peticiones HTTPS en producción (TLS 1.2 como mínimo). Toda petición a recursos protegidos deben incluir un token de sesión JWT válido y no expirado en la cabecera de "Authorization", las peticiones sin token o con uno inválido son rechazados.
- e) **Unidades:** Mensajes HTTP con cuerpo en formato JSON (UTF-8), códigos de estado HTTP estándar. Tiempo de respuesta medido en milisegundos, junto a un tamaño de payload

medido en bytes/kilobytes.

- f) **Timing:** *Igual que los definidos para INT-01.*
- g) **Relaciones:** *Es el canal que comunica a las interfaces INT-02, INT-03 con INT-01, INT-04 e INT-05 de forma interna por el servidor y no exponen comunicación HTTP directa al ciudadano.*
- h) **Formato de pantalla:** *No comprende UI.*
- i) **Formato de ventana:** *No comprende UI.*
- j) **Formato de datos:** *Igual que los definidos para INT-01.*
- k) **Formato de comandos:** *Igual que los definidos para INT-01.*
- l) **Mensajes de fin:** *No comprende mensajes, solo códigos de estado.*

3.2 Requerimientos Funcionales

- **RF1:** Propuesta de Normativa Legislativa por parte de "Colectivos Civiles": el sistema deberá permitir a los usuarios registrados como "Colectivos Civiles" redactar una propuesta normativa, facilitando la incorporación de recursos de soporte (PDFs, enlaces) y la edición hasta su publicación.
- **RF2:** Motor de Recolección y Validación de Firmas al igual que el anterior, el sistema permitirá a los ciudadanos realizar la firma digital de la propuesta, garantizando que cada firma sea única por ciudadano y por propuesta.
- **RF3:** Activación de Umbrales y Contador: el sistema iniciará un contador regresivo de 90 días calendario y un contador progresivo de firmas al publicar la propuesta.
- **RF4:** Cierre y Congelamiento Criptográfico: al conseguir exactamente 25,000 firmas válidas dentro del plazo establecido, el sistema debe parar la recepción de firma mediante el congelamiento y generar un hash criptográfico (ej. SHA-256) de todo el expediente (propuesta, recursos y listado de firmas) firmado con un certificado digital del Estado.
- **RF5:** Integración y Despliegue Legislativo: al congelarse, el sistema empaquetará el expediente y lo enviará mediante una API de forma segura en la bandeja de entrada de la Mesa de Partes u Oficina del Congreso para su derivación a comisiones.

3.3 Requerimientos de usabilidad

- **RU1:** Accesibilidad Universal (Cumplimiento WCAG 2.1 AA): La plataforma debe ser completamente navegable mediante teclado y compatible con lectores de pantalla (como NVDA o VoiceOver). Debe incluir opciones de alto contraste y tamaño de texto ajustable para garantizar la participación de ciudadanos con discapacidades visuales o motoras.
- **RU2:** Diseño Orientado a Dispositivos Móviles (Mobile-First): Dado que la gran mayoría de la población accederá a través de sus teléfonos, las interfaces de lectura de propuestas y, sobre todo, el botón de "Firma Digital", deben estar optimizados para pantallas táctiles (botones de al menos 44x44 píxeles) y resoluciones desde 320px de ancho.
- **RU3:** Fricción Mínima en el Flujo de Firma: El proceso desde que un ciudadano lee la propuesta hasta que emite su firma válida no debe requerir más de 3 clics o pasos. El sistema debe guiar al usuario visualmente durante la validación de su identidad sin llevarlo a callejones sin salida.
- **RU4:** Lenguaje Ciudadano y Apoyo Cognitivo: La interfaz debe utilizar un lenguaje claro y directo, evitando la jerga legislativa en los menús y botones de acción. Se deben implementar tooltips (cuadros de ayuda contextual) o un glosario integrado para explicar términos técnicos o legales presentes en las propuestas.

3.4 Requerimientos de rendimiento

- **RR1:** Latencia de la Transacción de Firma: La API de firma ciudadana debe responder en un máximo de 1.5 segundos. Continental, la API debe verificar la identidad del ciudadano con la base de datos externa (ej. RENIEC o registro civil), realizar la validación de duplicados y activar el contador.

- **RR2:** Alta Concurrencia (Efecto Viral): El sistema debe soportar picos de al menos 5.000 usuarios concurrentes visualizando y firmando la misma propuesta simultáneamente sin que la tasa de error (timeouts o HTTP 5xx) supere el 0,1%.
- **RR3:** Integridad Transaccional bajo Carga: El motor de base de datos debe procesar múltiples escrituras concurrentes garantizando operaciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad) bajo ninguna circunstancia de alta carga concurrente (race conditions), el contador debe realizar exactamente las 25.000 firmas antes del bloqueo.
- **RR4:** Procesamiento Criptográfico Asíncrono: Las operaciones de generación de hash SHA-256 y compresión de los documentos tras llegar a la firma 25.000 son operaciones intensivas en CPU, deben ser ejecutadas en un hilo de fondo (background job) y no exceder los 30 segundos, evitando bloquear la capacidad del servidor web para ofrecer otras propuestas.

3.5 Requerimientos de bases de datos

- **RBD-01:** La base de datos deberá almacenar la relación 1:N ciudadanos y sus reportes.
- **RBD-02:** El sistema de base de datos deberá persistir los metadatos del ciudadano e ISBN para cada registro.
- **RBD-03:** Al intentar eliminar registró, el motor de base de datos deberá verificar la inexistencias de otros registros hechos por el usuario anteriormente

3.6 Restricciones de diseño

- **RD-01:** El sistema deberá implementarse siguiendo una arquitectura RESTful, utilizando correctamente los métodos HTTP (GET, POST, PUT, DELETE) para las operaciones sobre los recursos.
- **RD-02:** El sistema deberá desarrollarse como una aplicación web compatible con navegadores modernos (Chrome, Firefox, Edge), sin requerir plugins adicionales.
- **RD-03:** El sistema deberá utilizar una base de datos relacional para la gestión de entidades de dominio, garantizando integridad referencial y consistencia de datos.
- **RD-04:** El sistema deberá ser desplegable en entornos de servidores estándar, sin necesidad de hardware especializado ni configuraciones propietarias.
- **RD-05:** La configuración del sistema (conexión a base de datos, puertos, credenciales) deberá ser externa y parametrizable, evitando dependencias rígidas en el código fuente.
- **RD-06:** El sistema no deberá incluir integraciones con servicios externos no especificados en el alcance del proyecto.
- **RD-07:** El sistema deberá utilizar el protocolo HTTPS para la comunicación en entornos de producción, garantizando la seguridad de los datos transmitidos.

3.7 Atributos del sistema de software

- **AS-01: Fiabilidad**
El sistema deberá garantizar la correcta ejecución de operaciones críticas (registro, tracking, etc) sin pérdida ni corrupción de datos, incluso ante fallos parciales del sistema.
- **AS-02: Disponibilidad**
El sistema deberá estar disponible para los usuarios al menos el 99% del tiempo, considerando mecanismos de recuperación ante fallos y reinicio automático del servicio.
- **AS-03: Seguridad**
El sistema deberá proteger la información contra accesos no autorizados mediante:
 - Autenticación basada en tokens JWT
 - Uso obligatorio de HTTPS
 - Sanitización de entradas para prevenir ataques como SQL Injection y XSS
- **AS-04: Capacidad de mantenimiento**

El sistema deberá estar diseñado de forma modular, permitiendo:

- *Separación clara entre frontend, backend y base de datos*
- *Facilidad para realizar cambios o correcciones sin afectar otros módulos*
- *Código documentado y estructurado*

- **AS-05: Portabilidad**

El sistema deberá poder desplegarse en distintos entornos (local, desarrollo, producción) sin modificaciones significativas en el código fuente, mediante:

- *Uso de tecnologías web estándar*
- *Configuración externa del entorno*
- *Independencia del sistema operativo del servidor*

4. Verificación

4.1 Interfaces externas

- **INT-01: API RESTful del sistema VozCi**

- *Envío de peticiones a todos los endpoints definidos verificando que se retornen códigos HTTP correctos y que las rutas que requieran autenticación rechacen cualquier otra petición.*
- *Envío de cadenas con patrones de inyección SQL y/o XSS en campos de entrada para confirmación de neutralización sin procesamiento del ataque.*
- *Medición del tiempo de respuesta del endpoint de búsqueda bajo carga de 500 usuarios concurrentes, verificando que el 95% de las respuestas se resuelvan en menos de 1 segundos o aproximación.*

- **INT-02: Portal web del cliente**

- *Navegación guiada y user-friendly entre las 5 vistas definidas (Página principal, catálogo, detalle de libro, mi perfil, historial de préstamos, registro e inicio de sesión) en navegadores modernos (Chrome, Firefox, Edge), verificando accesibilidad y renderización adecuada de portadas y datos del catálogo.*
- *Ingreso de texto límite en los recuadros correspondientes (barras de búsqueda, comentarios y descripciones de textos), asegurándose que el sistema acepte correctamente los datos brindados sin colapsar la memoria.*
- *Verificación del botón de reserva en diferentes escenarios dados por usuario: Usuario autenticado sin multas con libro disponible, usuario con multa pendiente, usuario no autenticado, etc.*
- *Medición del tiempo de carga de la página de inicio bajo conexión internet estable (~100 Mbps) tal que cumpla con lo acordado en la sección de "Timing".*

- **INT-03: Panel web del administrador**

- *Acceso con credenciales administrador por separado, verificando que cada rol accede únicamente a las vistas y acciones autorizadas para su perfil.*
- *Confirmación de devolución en el sistema que verifique el cambio de estado en el registro de incidencias y que los cambios de texto con entrada de caracteres excesiva sea truncada.*

- **INT-04: Base de datos relacional**

- *Revisión precisa de los diagramas Entidad-Relación y scripts SQL para confirmación de las relaciones pertinentes entre diferentes agentes.*
- *Solicitud de un objeto con clave inexistente, verificando que el sistema retorna un código de estado adecuado (404) y que la UI muestre un mensaje por defecto.*

- **INT-05: Servicio de almacenamiento de objetos**

- *Carga de archivos en formato válido (JPEG, PNG) y rechazo de formatos inválidos (PDF, GIF) y*

por encima del tamaño máximo definido, verificando que el sistema acepte los primeros y rechace los restantes o arroje un mensaje de aviso al usuario.

- **INT-06: Protocolo HTTP/HTTPS con intercambio JSON**
 - Inspección de la configuración del servidor para confirmar que solo se aceptan conexiones HTTP con TLS 1.2 o superiores y una revisión de cabeceras de seguridad pertinentes.
 - Verificación del cumplimiento de los contratos o esquemas para la estructura de datos de cada uno de los endpoints provocando distintos códigos de estado correspondientes.
 - Revisión del código del backend para confirmar que los verbos HTTP se usan de acuerdo a la semántica RESTful.

4.2 Requerimientos Funcionales

- **RF-01, RF-02:** Ejecución de casos de prueba para validar que los campos (como el ISBN o correos electrónicos) acepten datos válidos y rechacen entradas erróneas.
- **RF-03:** Escaneo del código fuente con herramientas automáticas para detectar vulnerabilidades como puntos de inyección de código o falta de sanitización.
- **RF-04:** Recorrido paso a paso por la interfaz web para verificar que el botón de registrar se habilite o deshabilite correctamente según sea el ciudadano
- **RF-05:** Simulación de caídas de red o errores de servidor durante una transacción para verificar que el sistema ejecute el rollback y no deje datos inconsistentes.
- **RF-06:** Ingreso de volúmenes de datos extremos (incidencias detalladas por personas quisquillosas)

4.3 Requerimientos de usabilidad

- **RU-01:** Realización de prueba con 3 usuarios reales con perfil de ciudadanos de una comunidad sin conocimiento del sistema pidiéndoles completar el registro de una incidencia sin ayuda externa.
- **RU-03:** Provocar deliberadamente los principales escenarios de error del sistema desde la interfaz de usuario (ISBN inválido, campos vacíos, intento de reserva con multa pendiente, sesión expirada, libro no disponible) y evaluar cada mensaje mostrado.
- **RU-05:** Inspección visual recorriendo todas las vistas definidas del sistema en al menos dos navegadores distintos, verificando que colores, tipografías, iconos y patrones de navegación (menú, botones, formularios) son homogéneos entre vistas y no presentan inconsistencias visuales ni funcionales entre ellas.

4.4 Requerimientos de rendimiento

- **RP-03:** Uso de scripts o frameworks de prueba para simular hasta 500 usuarios navegando simultáneamente y medir si el tiempo de respuesta se mantiene bajo los 3 segundos.
- **RP-02:** Medición del tiempo de carga de la interfaz para asegurar que el contenido crítico aparezca en menos de 2.5 segundos.
- **RP-03:** Evaluación del consumo de recursos del servidor bajo carga máxima para asegurar que no haya fugas de memoria o cuellos de botella.

4.5 Requerimientos de bases de datos

- **RBD-01:** Revisión técnica del diagrama de Entidad-Relación y de los scripts SQL para confirmar que las relaciones reflejen la lógica del negocio.
- **RBD-01**
- **RBD-03:** Comprobación tras un reinicio forzado del servidor para asegurar que todos los metadatos y estados de disponibilidad se hayan guardado correctamente.

4.6 Restricciones de diseño

- Revisión del código fuente para comprobar que la arquitectura implementada sigue el estilo RESTful mediante el uso correcto de métodos HTTP.
- Validación del funcionamiento del sistema en navegadores web modernos para asegurar

compatibilidad.

- *Despliegue del sistema en un entorno de servidor estándar para verificar que no requiere hardware especializado.*
- *Inspección de la configuración del sistema para confirmar que las conexiones a la base de datos son configurables.*
- *Verificación de que no existen integraciones con servicios externos no contemplados en el diseño.*

4.7 Atributos del sistema de software

- *Ejecución de pruebas de seguridad (inyección SQL, XSS) para validar los mecanismos de protección del sistema.*
- *Simulación de fallos del sistema o caídas del servidor para verificar la capacidad de recuperación y disponibilidad.*
- *Ejecución repetida de operaciones críticas (préstamos, devoluciones) para asegurar la fiabilidad del sistema.*
- *Revisión del código fuente para evaluar la modularidad y facilidad de mantenimiento.*
- *Despliegue del sistema en distintos entornos para comprobar su portabilidad y correcto funcionamiento.*

5. Apéndices

5.1 Suposiciones y dependencias

- Se asume que la entidad que adopte Inci(el seridor de una municipalidad posiblemente) dispone de un servidor estándar con al menos 4 GB de memoria RAM para alojar el sistema, sin necesidad de hardware especializado.
- Se asume que los ciudadanos redacten bien su reporte de manera que sea lo más legible posible.
- Se asume que el volumen máximo de usuarios simultáneos bajo condiciones normales de operación no superará los 500, cifra empleada como base para los requerimientos de rendimiento definidos en la sección 3.4.
- Se asume que el sistema operará bajo el protocolo seguro de comunicación en entornos de producción, siendo responsabilidad de la entidad desplegente la provisión y renovación del certificado de seguridad correspondiente.
- El sistema depende de una base de datos relacional compatible con el lenguaje de consulta estándar para la persistencia de todas las entidades de dominio definidas en la sección 1.4.
- El despliegue del servicio de almacenamiento de objetos es opcional, pero si se activa, el sistema depende de un servicio compatible con protocolos estándar de almacenamiento de objetos para la gestión de imágenes y recursos multimedia.
- El portal web del cliente y el panel de administración dependen completamente de la disponibilidad de la interfaz de programación del sistema; una caída del backend implica la inoperabilidad de ambas interfaces de usuario.
- La autenticación del sistema depende de la correcta configuración del mecanismo de generación y verificación de los tokens de sesión, incluyendo su tiempo de expiración configurable.

5.2 Siglas y abreviaturas

- **REST / RESTful** – *Representational State Transfer*
- **HTTP / HTTPS** – *Hypertext Transfer Protocol / Hypertext Transfer Protocol Secure*
- **JSON** – *JavaScript Object Notation*
- **ISBN** – *International Standard Book Number*
- **SQL** – *Structured Query Language*
- **SQLi** – *SQL Injection*
- **XSS** – *Cross-Site Scripting*
- **PDF** – *Portable Document Format*
- **TLS** – *Transport Layer Security*
- **UUID** – *Universally Unique Identifier*
- **CRUD** – *Create, Read, Update, Delete*
- **SRS** – *Software Requirements Specification*

- **UI** – User Interface, interfaz de usuario.
- **URL** – Uniform Resource Locator
- **UTF-8** – Unicode Transformation Format de 8 bits
- **ER** – Entidad-Relación
- **IEEE** – Institute of Electrical and Electronics Engineers
- **ISO / IEC** – International Organization for Standardization / International Electrotechnical Commission
- **RF** – Requerimiento Funcional
- **RU** – Requerimiento de Usabilidad
- **RP** – Requerimiento de Rendimiento
- **RBD** – Requerimiento de Base de Datos, identificador usado en la sección 3.5.
- **RD** – Restricción de Diseño, identificador usado en la sección 3.6.
- **AS** – Atributo del Sistema de software, identificador usado en la sección 3.7.
- **INT** – Interfaz, identificador usado en la sección 3.1